Buying back communication with retrieval compute in federated fine-tuning

Jeonghun Kim · Macquarie University

STEM research placement · 16 September – 28 October 2026

Placement deliverable: report

Abstract

Communication dominates the cost of federated fine-tuning, and client-side retrieval is often proposed as a way to reduce it: if clients train on better documents they should reach a target accuracy in fewer rounds, and rounds are bytes. This placement tested that proposition directly. Twenty-four federated runs of SmolLM2-135M with LoRA adapters were trained on Natural Questions under an identical communication budget of 1.74 GB, varying only the cross-encoder reranking depth d over six levels (0, 3, 7, 10, 25, 50) and four seeds. The treatment took effect as designed: the share of prompt passages that dense retrieval alone would not have selected rose monotonically from 0 % to 52.5 %. Downstream learning did not follow. Rounds-to-target was measured at 63 thresholds derived from the baseline curves rather than at one chosen value, and no depth resolved a saving at more than 5 of the 63. The largest median saving belonged to $d = 3$, an ordering control that cannot change which documents appear. A two-term cost model converted the remaining differences into an exchange rate of bytes per FLOP; eight of ten rates were negative and both positive ones belonged to the control. A mechanism check located the break: mean client-update divergence moved by 0.77 % while treatment strength rose from 0 to 0.72. Better documents reached the clients; the clients did not use them. The design's own resolution limit is reported alongside the result — with four seeds a paired permutation test cannot reach $p < 0.05$ at all — so the finding is that the fewer-rounds path alone cannot buy back the reranking compute at this model scale, not that reranking is uneconomic.

1. Introduction

Federated learning lets many clients train a shared model without pooling their raw data (McMahan et al., 2017). Its dominant operating cost is communication: every round, every participating client uploads and downloads a parameter update, and the number of rounds multiplies that cost. Parameter-efficient fine-tuning reduces the size of each message — a LoRA adapter is a small fraction of a model's weights (Hu et al., 2022) — but it does not reduce the number of messages.

A separate line of work argues that the quality of the documents a model trains on changes what it learns. Retrieval-augmented fine-tuning shows that varying the composition of retrieved context during training changes downstream accuracy (Zhang et al., 2024). Putting the two together

suggests an attractive trade: spend local compute to retrieve better documents, learn faster, finish in fewer rounds, and pay fewer bytes. Local compute is comparatively cheap; bandwidth, in the federated setting, is not.

This placement asked whether that trade actually closes.

> ***Research question.*** *How much communication can be bought back by spending client-side retrieval compute in federated LLM fine-tuning, and at what exchange rate?*

The hypothesis was a chain of four links: raising reranking depth puts better documents in the prompt; better documents make clients learn the same task more consistently; more consistent clients need fewer rounds; fewer rounds means fewer bytes. The objectives followed from the chain. First, vary one axis — reranking depth — while holding the communication budget identical, so that any difference in rounds-to-target is attributable to the treatment. Second, measure the saving without choosing a target accuracy, since the choice of target is otherwise free to determine the answer. Third, price the saving against measured reranking FLOPs so the verdict is a quantity with units rather than a direction. Fourth, if the chain fails, identify which link broke.

The fourth objective turned out to be the one that mattered.

---

2. Background and related work

Trading compute for rounds is the original federated idea. McMahan et al. (2017) introduced FedAvg precisely on the argument that on-device computation is cheap relative to communication, so extra local work should be spent to reduce the number of rounds. Their mechanism was more local steps per round. A survey of federated learning in edge networks reports that this works well when client data is identically distributed and much less well when it is not — for MNIST CNNs, more local computation cut rounds by over thirty times on IID data but only 2.8 times on non-IID data (Lim et al., 2020). The present study varies a different axis, retrieval quality rather than local epochs, under a non-IID partition.

Federated retrieval-augmented generation exists, but optimises the opposite direction. RAGRoute routes each query to a subset of federated sources, reducing how much reranking is done and cutting communication volume by about 80 % at query time (Guo et al., 2025). That is reranking as a cost to be minimised. The question here is the inverse: whether paying *more* reranking cost during training buys back enough training communication to be worth it.

Document composition during training does affect accuracy. RAFT trains with a controlled mixture of oracle and distractor documents and reports that the mixture changes test-time performance (Zhang et al., 2024). This supports the second link of the chain in principle, though it is measured on accuracy at much larger model scales and is not connected to communication.

Order is not neutral. Liu et al. (2024) show a U-shaped positional effect: the same documents in a different order produce different answers, with material in the middle of a context used least

effectively. This result directly shapes the design below, because it means a control that reorders passages without changing them is not a control that changes nothing.

No study was found that varies retrieval quality as a treatment and measures federated rounds-to-target and communication bytes as the response. The literatures are adjacent and unconnected, which is the gap this placement occupies.

---

3. Materials

3.1 Model

All runs fine-tuned `HuggingFaceTB/SmolLM2-135M-Instruct`, a 135 M-parameter decoder-only model from the SmolLM2 family, in bfloat16. Only LoRA adapters were trained: 921,600 trainable parameters, 0.68 % of the base model. The base model was fetched from the Hugging Face Hub at run time and is not redistributed in the archive.

The checkpoint was not assumed to be adequate. A capability-headroom gate (Section 4.3) measured a two-shot F1 of 19.57 with gold context against 1.17 with random context, a gap of 18.40 F1 against a pre-declared minimum of 5.0. There was therefore room for better retrieval to express itself, had the model been able to use it.

3.2 Question set and passage corpus

Questions came from `google-research-datasets/nq_open`, the open-domain variant of Natural Questions (Kwiatkowski et al., 2019; Lee et al., 2019), with 87,925 training and 3,610 validation items. Passages came from a parquet mirror of the DPR Wikipedia split, `kenhktsui/wiki_dpr_e5`, whose identifiers and text match `psgs_w100` (Karpukhin et al., 2020); the canonical `wiki_dpr` loader no longer works under `datasets` ≥ 3, which is why the mirror was used. Only the `id`, `title` and `text` columns were read, which cut the download from roughly 86 GB to 8 GB by avoiding the attached embedding column.

A 200,000-passage working corpus was built from the 21 M-passage source. Gold passages were identified by a two-stage rule: a normalised answer string had to be contained in the passage, *and* the passage had to share at least two discriminative content words with the question. The second stage is not cosmetic. Containment alone marks 98 % of the corpus as somebody's gold passage, because 69 % of Natural Questions answers are a single token such as "1950" or "North"; requiring two overlapping content words removed 96.8 % of those false positives. Each question additionally received up to five BM25 hard negatives, with answer-bearing candidates excluded so that a distractor could not be accidentally correct. The corpus scan covered 84,076 of 91,535 questions.

3.3 Clients and partitions

Eight clients were simulated with full participation. Each held 10,090 questions. Two partition assignments (seeds 1001 and 1002) were alternated across experimental seeds so that no result would be an artefact of one allocation; this is why the seed count must be even, and why four rather

than five seeds were run.

---

4. Methods

4.1 Treatment axis and the ordering control

Every condition placed exactly three passages in the prompt. The only thing that varied was how many candidates were examined before those three were chosen. Dense retrieval produced a candidate list; at depth `d` the cross-encoder `cross-encoder/ms-marco-MiniLM-L-6-v2` (Nogueira and Cho, 2019) scored the top `d` candidates and the best three were kept.

Six depths were run: 0, 3, 7, 10, 25, 50. Depths 7 and 25 were added part-way through the placement in response to mentor feedback that the ladder was too coarse to show a trend.

`d` = 3 is the ordering control. Choosing three out of three cannot admit a fourth passage, so the set of documents is fixed by construction and only their arrangement changes. Two quantities were logged per condition to separate these effects: **promotion rate**, the share of prompt slots holding a passage dense retrieval would not have selected, and **rank-change rate**, the share of slots holding a different passage than the baseline, which includes mere reordering. Their difference is the reshuffling component.

Following Liu et al. (2024), `d` = 3 is *not* treated as a condition in which nothing changed, because reordering can itself move the score. It is treated as a condition in which the *documents* cannot improve. That distinction governs the interpretation in Section 6.

4.2 Identical communication budget

Total communication was computed as

```
R x N x (|A| + |B|) x bytes-per-parameter x 2
```

where `R` is rounds, `N` clients, and `|A| + |B|` the LoRA adapter parameter count — a sum of the two low-rank factors, not a product. At 921,600 parameters accounted in fp16 this is 1.843 MB per client per direction, 29.49 MB per round, and 1.74 GB over 59 rounds. Every run spent exactly this; a post-hoc integrity check confirmed an identical budget across all 24 runs, and early stopping was disabled so that no conclusion could depend on a stopping rule.

4.3 Pilot gates

Three gates were run before the main sweep, each with a pre-declared pass criterion, so that a null result could be attributed to the treatment rather than to an unsuitable setup.

Gate 1, capability headroom. Reported in Section 3.1. Passed.

Gate 2, treatment monotonicity. On 2,000 held-out questions, gold recall at 3 rose from 0.467 at `d` = 0 to 0.5095 at `d` = 50. The gate also measured the share of questions on which the treatment *could* act — excluding those whose gold passage was already in the top three and those whose gold

passage lay outside the candidate pool entirely. That recoverable share was 0.202 to 0.307 against a declared minimum of 0.10. Passed.

Gate 3, local-step calibration. Client-update divergence was measured across candidate local-step counts $K \in \{4, 8, 16, 32, 64\}$ using nested snapshots along one trajectory per round. The probe, run at 250 questions per client, pointed to $K = 4$; the decision was re-derived at full scale, where $K = 32$ gave the lowest round-one divergence (0.3343) at a spread ratio of 4.31 against the smallest K. The schedule was fixed at $K = 32$, $S = 1{,}892$ total local steps, $R = 59$ rounds, which is 1.5 passes over each client's data at an effective batch of 8. Passed.

4.4 Evaluation held at the baseline depth

Every model, whatever depth it was trained at, was **scored on `d` = 0 documents**. Logs confirm that evaluation document quality was identical to three decimal places across conditions.

This matters because reranking can raise a score by two different routes. The *training* route — better documents produce a better model — reduces rounds and therefore bytes. The *serving* route — a better-equipped model at test time — does not reduce rounds at all, and requires reranking to run on every query forever. Scoring each condition at its own depth would sum the two routes and make them indistinguishable, and the serving route almost certainly lifts the score: at $d = 50$ the evaluation-time document quality was 3.92 pp above $d = 0$. Mixing it in would have manufactured a communication saving where none existed.

The price of this choice is stated plainly: the reported scores are lower than a deployed system would show, and the serving benefit is credited at zero throughout the cost model. The accounting is deliberately one-sided and unfavourable to reranking. Section 8 records this as a limitation on the scope of the conclusion.

4.5 Frozen artefacts

Corpus construction, client partitions, dense retrieval results and cross-encoder scores were computed once and frozen before the main sweep. The reranking cache in particular means each question was scored once per depth rather than once per round. Depth therefore selects a different slice of the same frozen artefacts, which removes retrieval stochasticity as a source of between-run variation and makes the 24 runs differ in exactly one respect.

---

5. Statistical methods

5.1 Rounds-to-target, swept rather than chosen

The primary measure was the number of federated rounds required to first reach a target held-out F1, read by linear interpolation between evaluated rounds. Interpolation was performed **per seed and then averaged**, never the reverse: averaging the curves first destroys the between-seed variance that the interval depends on.

Choosing a single target would let that choice determine the result. Instead the target τ was swept over the baseline curves' own evaluated values, with both bounds derived rather than declared: the lower bound is the maximum across seeds of the baseline's round-one score (18.13), since nothing below it is reachable in a whole round, and the upper bound is the minimum across seeds of the baseline's peak (28.14), above which some seeds never arrive. Of 80 candidate values, 3 fell below the lower bound and 14 above the upper, leaving **63 targets**.

5.2 Resolution, and what this design can detect

Each depth was compared with the baseline **paired by seed**, because seed identity shifts a whole run's round count up or down and pairing cancels that component: the standard error of a difference is 0.16–0.47 rounds paired against 0.879 unpaired.

A saving was counted as resolved only when it exceeded the two-tailed 95 % threshold for its own standard error. With four seeds the relevant critical value is **t(0.975, 3) = 3.182**, not 2 — an earlier draft of this work used twice the standard error, a rule 1.6 times too lenient, under which one contrast read as significant that is not ($d = 10$ vs $d = 0$, +0.60 ± 0.28, $p = 0.127$). The threshold works out at 0.52–1.48 points depending on the pair, against a largest observed difference of 0.60.

The design's ceiling is reported with the result. In a paired design the only distribution-free permutation is a sign flip per seed, giving $2^4 = 16$ arrangements and a smallest attainable two-tailed p of **0.125**. With four seeds, $p < 0.05$ is structurally unreachable however clean the data; six seeds would bring the floor to 0.031. The smallest p actually observed was 0.127, essentially on that floor. Detecting an effect at 80 % power would have required it to be 0.67–1.96 points; the observed effects were 0.03–0.60 points.

5.3 A cross-check that uses no interpolation

Because the interpolation itself could be doubted, a second reading fixed the axis the other way: at each evaluated round, with the bytes spent identical by construction, the difference in score between each depth and the baseline was taken, and a difference was called separated from zero when its magnitude exceeded twice its standard error. This reading shares no machinery with Section 5.1 and is reported alongside it.

5.4 Cost model

Reranking cost was decomposed into two terms:

```
one-off     indexing   = corpus size x FLOPs per passage
recurring   reranking  = Q x depth x FLOPs per pair
```

FLOPs per pair (5.437×10^4) and per passage (5.437×10^4) were measured with a FLOP counter, padded to the 256- and 128-token lengths the analytic estimate assumes so the two are comparable; measured and analytic agreed within a factor of 0.89 and 0.94 respectively. The indexing term is identical across depths and therefore cannot separate conditions, which is the reason depth rather than encoder capacity was chosen as the treatment axis: depth is a *recurring* per-query cost,

commensurable with per-round communication.

`Q`, the number of questions reranked, was reported at two values rather than left as a free symbol. The **measured** value is 80,720 — 10,090 questions per client across 8 clients, each reranked once because of the cache — and answers "did this experiment's compute pay for itself". The **declared** sweep, 10^3 to 10■, answers "would it pay for itself if deployed"; deployment scale cannot be measured by an experiment that was never deployed, so it is swept rather than chosen. Both are reported and they agree to within 24 %.

The exchange rate is bytes saved divided by FLOPs spent. Break-even bandwidth puts both in seconds, `B* = saved × 8 × throughput ÷ FLOPs`, at a declared device throughput of 10^{12} FLOP/s. Two indexing assumptions were carried throughout — index already held, index built for this study — and both were reported, though the sign of the rate is set by the numerator alone and no indexing assumption can change it.

5.5 Software

Python 3.11; PyTorch 2.14.0 (Paszke et al., 2019); Transformers 5.17.0 (Wolf et al., 2020); PEFT 0.21.1 (Mangrulkar et al., 2022); Flower 1.39.0 with simulation (Beutel et al., 2020); sentence-transformers 6.1.0; NumPy 2.4.6 (Harris et al., 2020); SciPy 1.17.1 (Virtanen et al., 2020); Matplotlib 3.11.2 (Hunter, 2007); pytest 9.1.1. Exact pins are in `requirements.txt`. All analysis scripts are archived with the data (Section 10) and the analysis path is covered by 333 automated tests.

---

6. Results

6.1 The treatment took effect

Promotion rate rose monotonically with depth and was exactly zero for both `d` = 0 and the `d` = 3 control, as the design requires.

d	Promotion rate	Rank-change rate	Difference
0	0.0 %	0.0 %	—
3 *(control)*	**0.0 %**	45.0 %	45.0 pp
7	36.2 %	64.8 %	28.6 pp
10	42.0 %	67.6 %	25.6 pp
25	49.8 %	71.2 %	21.4 pp
50	**52.5 %**	72.4 %	19.9 pp

Two things follow. The apparatus worked: deeper reranking really did place different documents in the prompt. And the effect saturates — a five-fold increase from `d` = 10 to `d` = 50 raised promotion by only a quarter, because the passages a cross-encoder promotes sit near the front of the candidate list. Cost, meanwhile, rises in proportion to depth. The gap between the two columns also narrows

with depth, which says that shallow reranking is largely reshuffling and only deep reranking is mostly substitution.

6.2 Final accuracy under the fixed budget

d	F1 (mean ± s.e.)	EM (mean ± s.e.)
0	29.13 ± 0.69	22.32 ± 0.63
3 *(control)*	29.44 ± 0.76	22.43 ± 0.64
7	29.20 ± 0.76	22.15 ± 0.62
10	29.72 ± 0.55	22.70 ± 0.47
25	29.24 ± 0.39	22.25 ± 0.35
50	29.41 ± 0.48	22.45 ± 0.46

All six conditions lie within about half a point of one another, and the spread across depths is smaller than the standard error of any one of them.

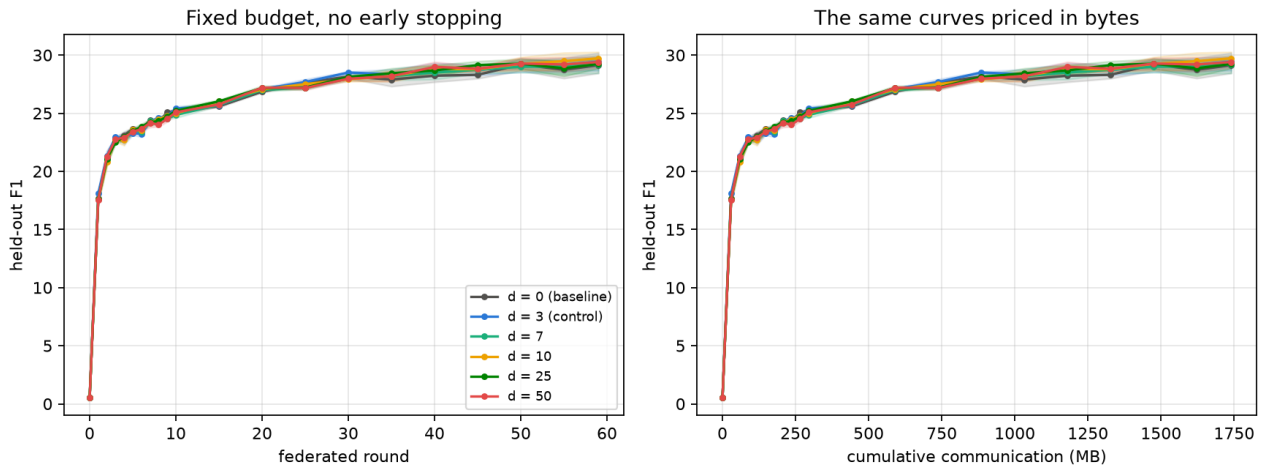


Figure 1. Held-out F1 against federated round (left) and against cumulative communication in megabytes (right), averaged over four seeds with standard-error bands. The two panels carry the same information because the budget is identical across conditions: 29.49 MB per round for all six depths. The curves are indistinguishable from one another for the whole run and have not flattened by round 59.

6.3 Rounds-to-target across 63 thresholds

d	Targets beaten	Of those, past threshold	Median
3 *(control)*	**41 / 63**	**5 / 63**	**0.15 rounds faster**
7	19 / 63	0 / 63	0.56 rounds slower
10	20 / 63	1 / 63	0.21 rounds slower
25	24 / 63	1 / 63	0.11 rounds slower
50	15 / 63	5 / 63	0.37 rounds slower

With no effect at all the win rate would sit near 50 %, about 31 of 63. Only the control exceeds that. Every condition that actually changed the documents falls below chance, and all four have negative

medians. No condition clears the decision threshold at more than 5 of the 63 targets.

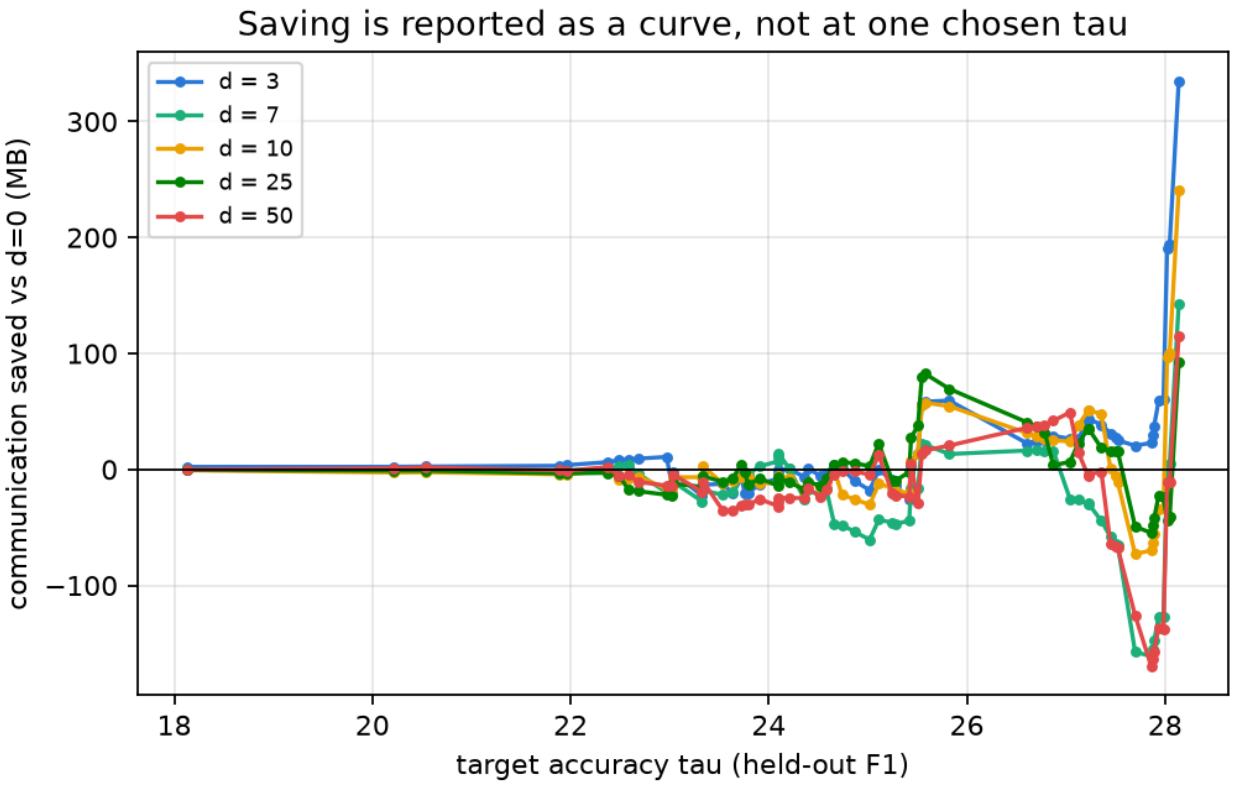


Figure 2. Communication saved against the baseline, in megabytes, at each of the 63 target thresholds. Saving is reported as a curve because reading it at a single chosen target would let that choice determine the result. The curves cross zero repeatedly and swing by more than ±150 MB at high targets, where the baseline is flat and a small score difference translates into many rounds. That tail was not trimmed, since choosing a cut point would be another arbitrary decision; extreme-insensitive statistics were used instead.

6.4 The interpolation-free cross-check agrees

Under the matched-budget reading, the control separated from zero at 8 of 20 evaluated rounds — the same count as d = 10, and more than any other condition. The two readings share no machinery and reach the same verdict.

6.5 Exchange rate and break-even

With the numerator taken as the median saving across all 63 targets, converted at 29.49 MB per round:

d	Bytes saved	Rate, index held	Rate, index built
3 *(control)*	**+4.41 MB**	**+2.70 × 10■■**	**+1.62 × 10■■**
7	−16.52 MB	−4.34 × 10■■	−3.38 × 10■■
10	−6.27 MB	−1.15 × 10■■	−9.61 × 10■¹■
25	−3.32 MB	−2.45 × 10■¹■	−2.26 × 10■¹■
50	−11.02 MB	−4.05 × 10■¹■	−3.90 × 10■¹■

Rates are quoted at `Q` = 100,000. A negative rate means compute was spent *and* more bytes were spent. Eight of the ten cells are negative, and the two positive cells both belong to the control. Every arm that actually changed the documents is negative. At the measured `Q` = 80,720 the picture is the same: break-even bandwidth is 0.027 Mbps for the control and exactly zero for all four treatments, against 0.022 and zero at the deployment value.

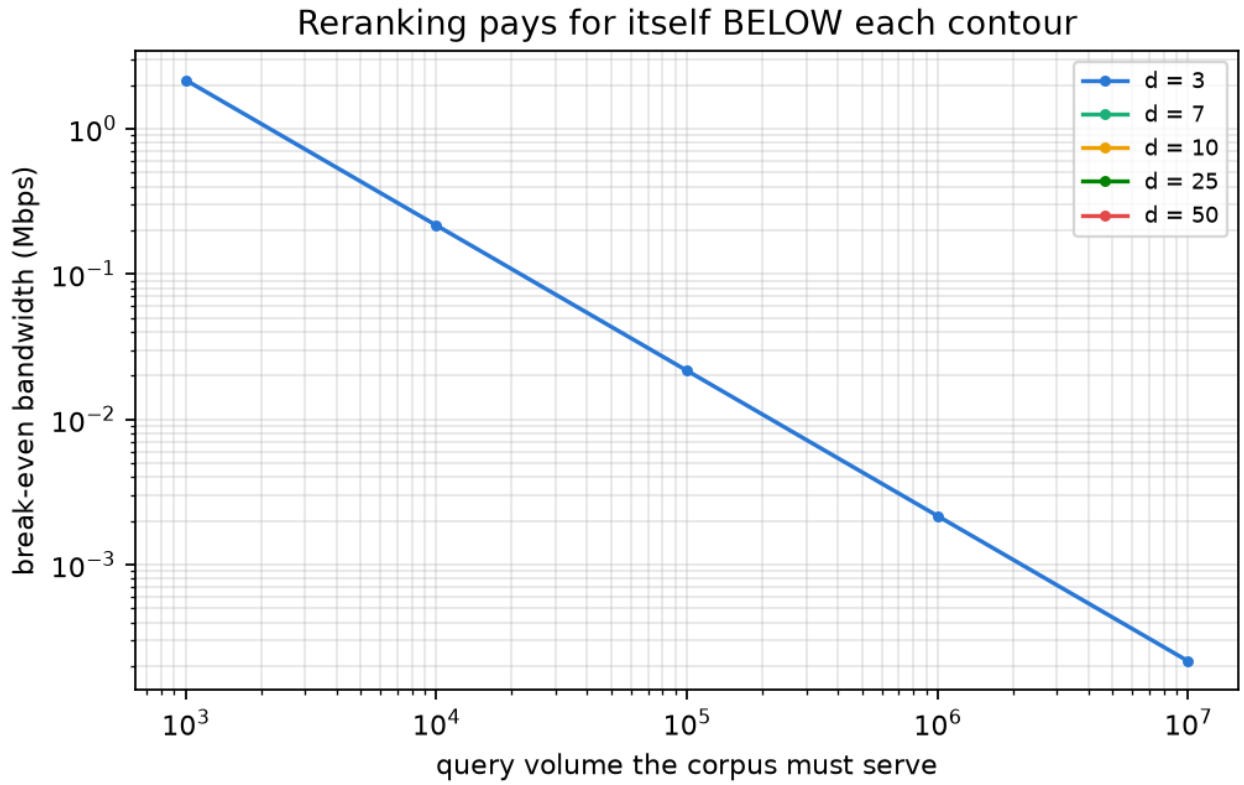


Figure 3. Break-even bandwidth against query volume. Reranking pays for itself only on links slower than the plotted contour. Four of the five depths have no contour at all, because their bytes saved are negative and a negative break-even bandwidth does not exist; they are reported as zero. The one contour belongs to `d` = 3, the control that cannot change which documents are retrieved.

The single non-zero contour reaches 2.161 Mbps at `Q` = 1,000, which does fall inside the declared 1–1,000 Mbps range. A thousand-query pilot on a link slower than about 2 Mbps would, arithmetically, break even — using the arm that retrieves nothing new.

6.6 Where the chain broke

d	Treatment strength	Client-update divergence
0	0.00	1.3234
3	0.45	1.3218
7	0.65	1.3143
10	0.68	1.3151
25	0.71	1.3162
50	0.72	**1.3132**

Treatment strength moved from 0 to 0.72. Divergence moved by 0.77 %, and not monotonically: d = 7 sits below both d = 10 and d = 25.

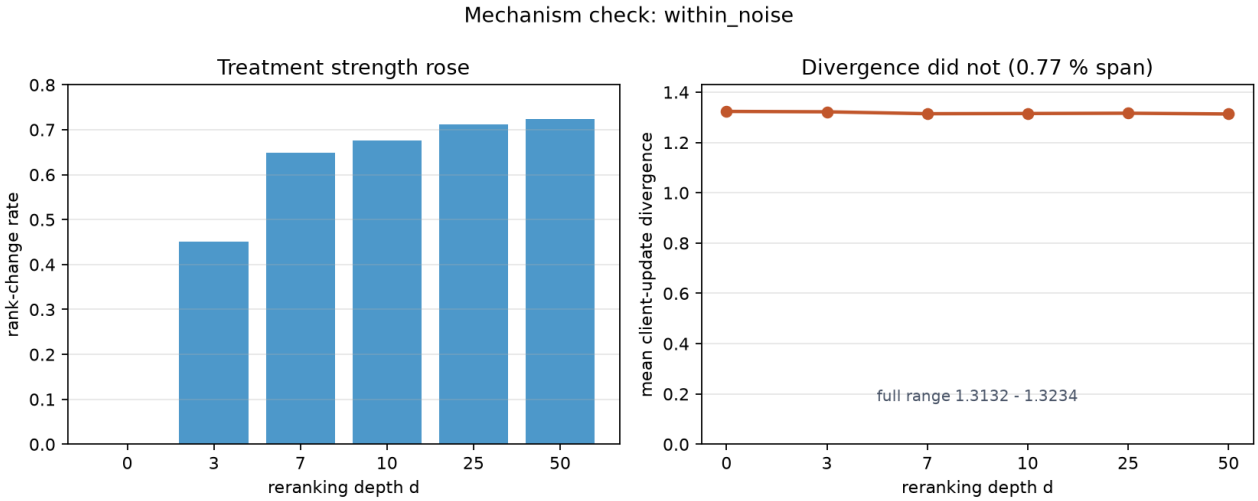


Figure 4. Mechanism check. Left: rank-change rate, the strength of the treatment actually applied to the prompt, which rises with depth. Right: mean client-update divergence over the first ten rounds, on an axis anchored at zero so that the magnitude of its movement is visible rather than magnified. The full range is 1.3132–1.3234, a span of 0.77 %. The treatment arrived; the learning dynamics did not respond.

The correlation between treatment strength and divergence reduction is +0.91, which in isolation looks like support. It must not be read that way, for two reasons. The correlation is computed over quantities that moved by 0.77 %, and a correlation coefficient reports direction without magnitude. And the next link has the wrong sign: divergence reduction against rounds saved is **−0.88**, where the hypothesis requires a positive value. The depths whose divergence fell most are the ones that spent the most extra rounds. There are also only five points, so coefficients of this size can arise by chance.

The verdict recorded by the analysis is `within_noise`, not `no_effect`: a saving counts as observed only when it clears the decision threshold at a majority of targets, and none does.

---

7. Discussion

7.1 The first link worked and the second did not

The chain can be scored link by link. Raising depth put better documents in the prompt: promotion rose monotonically to 52.5 %, and the apparatus checks confirm that nothing else varied. Better documents did not make clients learn more consistently: divergence barely moved. Fewer rounds did not follow: the control beat every treatment. And with no round saving there was nothing to convert into bytes.

The break is at the second link, and its most plausible cause is capacity. A 135 M-parameter model adapted through 921,600 LoRA parameters may simply lack the headroom to exploit a better context, even though Gate 1 confirmed the checkpoint can use gold context when handed it. The

distinction matters: the model can read a good document at inference time, but the gradient signal from training on a better document is apparently too small, relative to seed noise, to change how fast the federation converges. The better books reached the student; the student did not open them.

This is an interpretation, not a demonstration. It is the limitation listed first in Section 8 and the first experiment that should follow.

7.2 The depth trade-off

d	Compute per query	Promotion	F1 vs control	Rounds saved
0	0 s	0 %	–0.31	—
3 *(control)*	0.15 s	0 %	—	+0.15
7	0.34 s	36 %	–0.24	–0.56
10	0.49 s	42 %	+0.29	–0.21
25	1.23 s	50 %	–0.20	–0.11
50	2.46 s	53 %	–0.03	–0.37

Depth does not improve performance consistently: the sign of the F1 difference against the control alternates, and all four round savings are negative. Diminishing returns appear not in performance but in document quality, where a five-fold depth increase buys a quarter more promotion while compute rises seven-fold — the knee sits between $d = 10$ and $d = 25$. The best trade-off available in this experiment is $d = 0$: no depth returned what it cost, and $d = 50$ spends 2.46 s on every query in perpetuity while spending 11.0 MB more communication.

That recommendation holds inside this study's one-sided accounting. The compute column is reranking's deployment cost; its deployment benefit was set to zero by the design in Section 4.4. The defensible statement is that the fewer-rounds path alone cannot justify this compute, not that reranking should not be used.

7.3 Relation to prior work

The result is consistent with the IID/non-IID gap reported for FedAvg-style compute-for-rounds trades, where the benefit shrinks by an order of magnitude under heterogeneous data (Lim et al., 2020). It does not contradict RAFT, which measures accuracy at larger scales rather than rounds (Zhang et al., 2024). It is adjacent to federated RAG work such as RAGRoute, which treats reranking as a cost to minimise and reports that reranking dominates query latency (Guo et al., 2025) — a finding that points the same way as this one. Null results of this shape are not unusual in the area: fine-tuned models have been reported to underperform their baselines across RAG datasets, and pruning retriever training data has been reported to improve rather than degrade ranking quality.

---

8. Limitations

Model scale. One model, SmolLM2-135M. A model this size may lack the capacity to exploit better documents. This is the most likely explanation of the null and the first thing a follow-up should change.

Four seeds. The measurement moves by ±0.16 to ±0.47 rounds. More fundamentally, a paired permutation test over four seeds has a floor of $p = 0.125$, so no effect of any size could have been declared significant. Six seeds would lift that ceiling; resolving a 0.3-point effect at 80 % power would take roughly sixteen.

The curves had not converged. The budget was fixed at 59 rounds and most curves peak at round 50 or later (Figure 1), so this is a statement about the early- and mid-training regime.

Six depths. 5, 20 and 30 were not measured. Every observed effect (0.03–0.60 points) lies below the decision threshold (0.52–1.48 points), so a finer ladder could not read a quantity below the smallest division.

The control reorders. $d = 3$ cannot change the documents but does change their order, and its +0.31 ± 0.37 ($p = 0.46$) is an estimate of the ordering effect that is indistinguishable from zero. Isolating that effect would need a condition running the same three documents in several orders, which was not run. The conclusion does not rest on the unknown, because both $d = 0$ and $d = 3$ were used as baselines and all nine contrasts were non-significant under either.

One-sided accounting. Sections 5.4 and 6.5 charge reranking's deployment cost and credit its deployment benefit at zero. Taken far enough, this accounting defeats any one-off saving by construction, since B^* falls as 1/Q. The conclusion is therefore scoped to the training path.

Assumed device throughput. Break-even bandwidth scales with the assumed 10^{12} FLOP/s. Being a constant of proportionality it cannot change any sign, but the control's paying region scales with it; the measured CPU throughput during FLOP calibration was about 110 GFLOP/s, nine times lower, which would narrow that region further.

---

9. Conclusion

Reranking depth did not buy back communication in federated fine-tuning of a 135 M-parameter model under a 1.74 GB budget. The treatment demonstrably reached the clients — new-document share rose from 0 % to 52.5 % — but learning did not respond, client-update divergence moved by 0.77 %, and the arm that cannot change which documents appear outperformed every arm that can. Eight of ten exchange rates were negative and the two positive ones belong to that control.

The contribution is not the negative sign but its location. The chain breaks at "better documents produce better learning", which points a follow-up at model capacity rather than at retrieval depth. The result is also bounded honestly: with four seeds this design could not have produced a significant effect of any size, and the cost accounting was deliberately unfavourable to reranking. What can be said is that the fewer-rounds path alone does not pay for the reranking compute at this

scale — which is worth knowing, because that is the path the proposition rests on.

---

10. Data and code availability

All code, per-round logs and analysis scripts are archived on Zenodo and developed openly on GitHub. The archive contains `src/`, `tests/`, `scripts/`, `configs/`, `docs/`, the 24 per-round JSONL run logs with their summaries, the pilot-gate artefacts, the analysis JSON and the figures, together with `MANIFEST.sha256` for integrity verification and a frozen environment listing. Per-round model checkpoints are excluded as derived and bulky. The base model and both source datasets are fetched from the Hugging Face Hub at run time and are not redistributed.

Source code and analysis scripts are released under Apache-2.0. Experiment logs under `results/logs/` are released under CC BY-SA 4.0, because generated text in them reproduces material derived from a CC BY-SA source corpus.

The accompanying research-records document describes the data-management plan, the run inventory, provenance and the long-term storage arrangement in full.

---

11. Declaration on the use of generative AI

Generative AI was used as a coding and drafting assistant throughout this placement, in line with Macquarie University policy. Specifically: an AI assistant was used to write and refactor analysis code, to draft and revise report text, and to review the statistical reasoning. All experimental design decisions, the interpretation of results and the final text were reviewed and approved by the author. No experimental data were generated, altered or synthesised by an AI system; every number reported here is read directly from the archived run logs, and the analysis scripts that produce them are included in the archive so that any figure or table can be regenerated independently. The development history, including the reasoning behind each change, is preserved in the repository's commit record.

---

12. References

Beutel, D. J., Topal, T., Mathur, A., Qiu, X., Fernandez-Marques, J., Gao, Y., Sani, L., Li, K. H., Parcollet, T., de Gusmão, P. P. B., and Lane, N. D. (2020). Flower: a friendly federated learning research framework. *arXiv:2007.14390*.

Guo, R., et al. (2025). Efficient federated search for retrieval-augmented generation using lightweight routing. *arXiv:2502.19280*.

Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). LoRA: low-rank adaptation of large language models. *International Conference on Learning Representations*.

Hunter, J. D. (2007). Matplotlib: a 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.

Karpukhin, V., O■uz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., and Yih, W. (2020). Dense passage retrieval for open-domain question answering. *EMNLP*.

Kwiatkowski, T., Palomaki, J., Redfield, O., et al. (2019). Natural Questions: a benchmark for question answering research. *Transactions of the ACL*, 7, 453–466.

Lee, K., Chang, M.-W., and Toutanova, K. (2019). Latent retrieval for weakly supervised open domain question answering. *ACL*.

Lim, W. Y. B., Luong, N. C., Hoang, D. T., Jiao, Y., Liang, Y.-C., Yang, Q., Niyato, D., and Miao, C. (2020). Federated learning in mobile edge networks: a comprehensive survey. *IEEE Communications Surveys & Tutorials*, 22(3), 2031–2063.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2024). Lost in the middle: how language models use long contexts. *Transactions of the ACL*, 12, 157–173.

Mangrulkar, S., Gugger, S., Debut, L., Belkada, Y., Paul, S., and Bossan, B. (2022). PEFT: state-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft

McMahan, H. B., Moore, E., Ramage, D., Hampson, S., and Agüera y Arcas, B. (2017). Communication-efficient learning of deep networks from decentralized data. *AISTATS*, 54, 1273–1282.

Nogueira, R., and Cho, K. (2019). Passage re-ranking with BERT. *arXiv:1901.04085*.

Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: an imperative style, high-performance deep learning library. *NeurIPS*.

Rajpurkar, P., Zhang, J., Lopyrev, K., and Liang, P. (2016). SQuAD: 100,000+ questions for machine comprehension of text. *EMNLP*.

Virtanen, P., Gommers, R., Oliphant, T. E., et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272.

Wolf, T., Debut, L., Sanh, V., et al. (2020). Transformers: state-of-the-art natural language processing. *EMNLP: System Demonstrations*, 38–45.

Zhang, T., Patil, S. G., Jain, N., Shen, S., Zaharia, M., Stoica, I., and Gonzalez, J. E. (2024). RAFT: adapting language model to domain specific RAG. *arXiv:2403.10131*.